

Department of Computer Science and Mathematics CSC 447:

Parallel Programming and Multi Cluster Systems Fall24

Assignment 3

Parallelizing Mandelbrot Set

(Row-Wise Chunking and Load Balancing)

Performance Improvement in Matrix Multiplication

Using Matrix Transposition

Tala Hareb

PART 1

Code Review: The code generates a visual of the Mandelbrot set by iterating over a grid of pixels, mapping each to a complex number, and performing repeated calculations to determine if a point belongs to the set.

The program runs 10 trials and calculates the average processing time, outputting a grayscale image in PGM format (which we will transform into PNG for visualization).

Performance could be enhanced by introducing parallelization with OpenMP, allowing pixel calculations to be distributed across multiple threads. Additionally, precomputing complex number mappings could further optimize execution time.

1) Sequential Execution

- **Execution times** ranged from approximately 43 ms to 49 ms, with an **average execution time of 45.19 ms**.
- The generated image represents a **correct and stable rendering** of the Mandelbrot set, as expected.

Sequential output :

Execution time of trial [1]: 0.048587 seconds

Execution time of trial [2]: 0.044591 seconds

Execution time of trial [3]: 0.044582 seconds

Execution time of trial [4]: 0.049043 seconds

Execution time of trial [5]: 0.047652 seconds

Execution time of trial [6]: 0.043782 seconds

Execution time of trial [7]: 0.043282 seconds

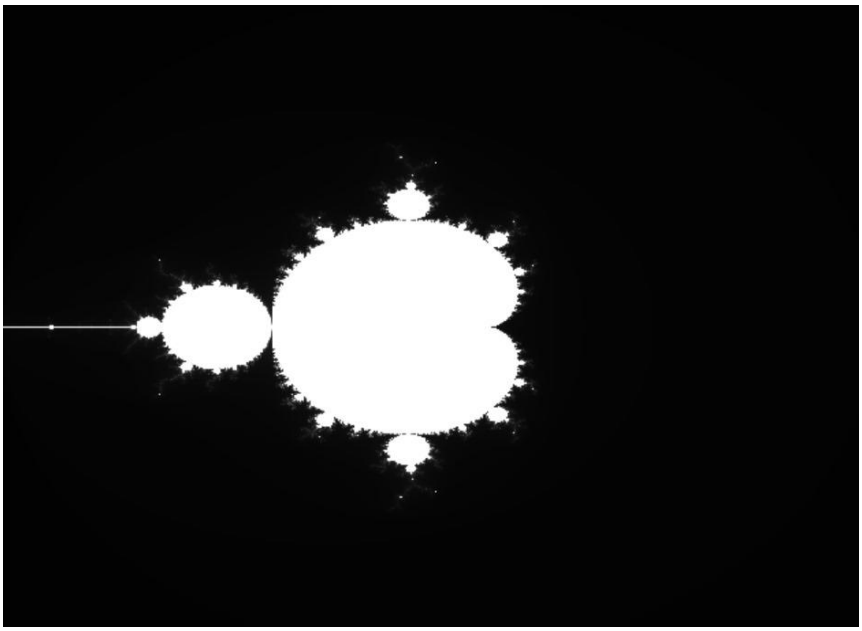
Execution time of trial [8]: 0.043583 seconds

Execution time of trial [9]: 0.043256 seconds

Execution time of trial [10]: 0.043499 seconds

The average execution time of 10 trials is: 45.185700 ms

Image:



2) Parallelization with Only private(c)

- Execution times were similar to the sequential ones, with slight variations in each run—sometimes a bit higher than 45 ms, sometimes a bit lower.
- However, the generated image was inaccurate due to **race conditions.**

Why Inaccurate Results Occurred:

- With only private(c), each thread had its own copy of the c variable, but j was shared across threads.
- This lack of privacy for j led to race conditions in the nested loop. Different threads accessed and modified j simultaneously, which resulted in incorrect values for c.real and c.imag.
- As a consequence, pixels were computed with the wrong complex numbers, leading to an inaccurate rendering of the Mandelbrot set.

Modified Code:

```
#include <stdio.h>
#include <time.h>

#define WIDTH 640
```

```
#define HEIGHT 480
#define MAX_ITER 255

#include <omp.h>

struct complex{
  double real;
  double imag;
};

int cal_pixel(struct complex c) {


            double z_real = 0;
            double z_imag = 0;

            double z_real2, z_imag2, lengthsq;

            int iter = 0;
            do {
                z_real2 = z_real * z_real;
                z_imag2 = z_imag * z_imag;

                z_imag = 2 * z_real * z_imag + c.imag;
                z_real = z_real2 - z_imag2 + c.real;
                lengthsq =  z_real2 + z_imag2;
                iter++;
            }
            while ((iter < MAX_ITER) && (lengthsq < 4.0));

            return iter;

}

void save_pgm(const char *filename, int image[HEIGHT][WIDTH]) {
    FILE* pgmimg;
    int temp;
    pgmimg = fopen(filename, "wb");
    fprintf(pgmimg, "P2\n"); // Writing Magic Number to the File
    fprintf(pgmimg, "%d %d\n", WIDTH, HEIGHT);  // Writing Width and Height
    fprintf(pgmimg, "255\n");  // Writing the maximum gray value
    int count = 0;

    for (int i = 0; i < HEIGHT; i++) {
        for (int j = 0; j < WIDTH; j++) {
```

```
            temp = image[i][j];
            fprintf(pgmimg, "%d ", temp); // Writing the gray values in the 2D
array to the file
        }
        fprintf(pgmimg, "\n");
    }
    fclose(pgmimg);
}

int main() {
    int image[HEIGHT][WIDTH];
    double AVG = 0;
    int N = 10; // number of trials
    double total_time[N];
    struct complex c;

    for (int k = 0; k < N; k++) {
        clock_t start_time = clock(); // Start measuring time
        int i, j;
       #pragma omp parallel for schedule(dynamic) private(c)
        for (i = 0; i < HEIGHT; i++) {
            for (j = 0; j < WIDTH; j++) {
                c.real = (j - WIDTH / 2.0) * 4.0 / WIDTH;
                c.imag = (i - HEIGHT / 2.0) * 4.0 / HEIGHT;
                image[i][j] = cal_pixel(c);
            }
        }

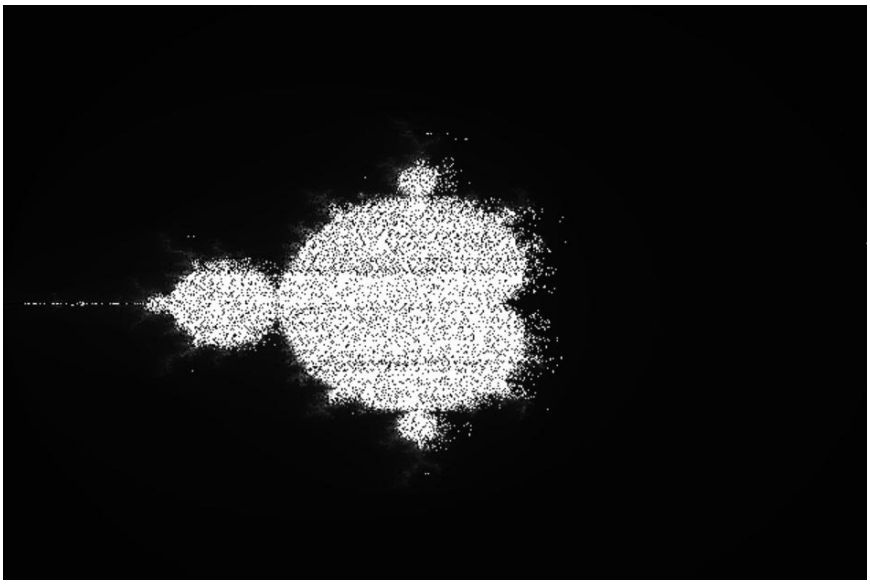
        clock_t end_time = clock(); // End measuring time

        total_time[k] = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
        printf("Execution time of trial [%d]: %f seconds\n", k + 1,
total_time[k]);
        AVG += total_time[k];
    }

    save_pgm("mandelbrot.pgm", image);
    printf("\nThe average execution time of %d trials is: %f ms\n", N, (AVG / N)
* 1000);

    return 0;
}
```

Image:



Output of code:

```
 tunix@VirtualAnimation:~/.cache/.vscode$ gcc tala.c -o tala -fopenmp
tunix@VirtualAnimation:~/.cache/.vscode$ ./tala
Execution time of trial [1]: 0.010392 seconds
Execution time of trial [2]: 0.044492 seconds
Execution time of trial [3]: 0.021583 seconds
Execution time of trial [4]: 0.062772 seconds
Execution time of trial [5]: 0.077357 seconds
Execution time of trial [6]: 0.004389 seconds
Execution time of trial [7]: 0.073393 seconds
Execution time of trial [8]: 0.013667 seconds
Execution time of trial [9]: 0.066709 seconds
```

Execution time of trial [10]: 0.046779 seconds

The average execution time of 10 trials is: 42.153300 ms

3) To get accurate results private(c,j)

we needed to make additional variables private. Specifically, **we chose to make j private instead of i**. Here's why:

- In the parallelized code each thread gets a unique value of i and handles it independently.
- However, **j is part of the inner loop** and is accessed multiple times within each iteration of the outer loop. Since j was shared, it led to conflicts discussed.
- By declaring j as private, we ensure each thread has its own independent copy of j, which eliminates any risk of interference or race conditions.

Modified Code main:

```
int main() {
    int image[HEIGHT][WIDTH];
    double AVG = 0;
    int N = 10; // number of trials
    double total_time[N];
    struct complex c;

    for (int k = 0; k < N; k++) {
        clock_t start_time = clock(); // Start measuring time
        int i, j;
       #pragma omp parallel for schedule(dynamic) private(c,j)
        for (i = 0; i < HEIGHT; i++) {
            for (j = 0; j < WIDTH; j++) {
                c.real = (j - WIDTH / 2.0) * 4.0 / WIDTH;
                c.imag = (i - HEIGHT / 2.0) * 4.0 / HEIGHT;
                image[i][j] = cal_pixel(c);
            }
        }

        clock_t end_time = clock(); // End measuring time

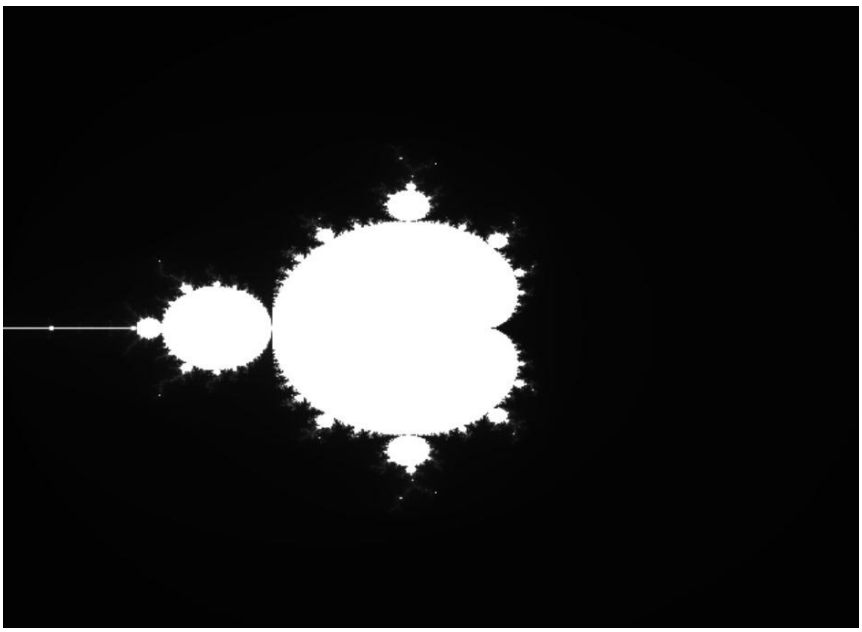
        total_time[k] = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
```

```
        printf("Execution time of trial [%d]: %f seconds\n", k + 1,
total_time[k]);
        AVG += total_time[k];
    }

    save_pgm("mandelbrot.pgm", image);
    printf("\nThe average execution time of %d trials is: %f ms\n", N, (AVG / N)
* 1000);

    return 0;
}
```

Image:



Output:

tunix@VirtualAnimation:~/.cache/.vscode$ gcc tala.c -o tala -fopenmp

tunix@VirtualAnimation:~/.cache/.vscode$ ./tala

Execution time of trial [1]: 0.059753 seconds

Execution time of trial [2]: 0.082232 seconds

Execution time of trial [3]: 0.067088 seconds

Execution time of trial [4]: 0.078271 seconds

Execution time of trial [5]: 0.080739 seconds

Execution time of trial [6]: 0.076933 seconds

Execution time of trial [7]: 0.082878 seconds

Execution time of trial [8]: 0.075825 seconds

Execution time of trial [9]: 0.079406 seconds

Execution time of trial [10]: 0.078861 seconds

The average execution time of 10 trials is: 76.198600 ms

4) Effect of private(c, j) on Accuracy and Execution Time

- Execution time increased by around 10%, with an average execution time of 76.20 ms.
- The results were now accurate, as the race conditions were fully eliminated by privatizing both c and j.

While adding the private directive for j corrected the results, it came at a performance cost.

- There's an increase in memory usage and a slight computational overhead from managing these independent variables.
- Furthermore, parallelization itself adds thread management overhead, which can sometimes offset the speed gains, especially with relatively small tasks like this image generation.

Summary of Findings

- Sequential Execution: Consistent results with an average execution time of 45.19 ms,
- Parallelization with Only private(c): Similar execution time on average but produces inaccurate results due to race conditions.

- Parallelization with private(c, j): Resolves the race condition, resulting in accurate results but with an increased execution time.

Threads

Main with specified number of threads: with private (c)

```
int main() {
    int image[HEIGHT][WIDTH];
    int thread_counts[] = {2, 4, 8}; // Array of different thread counts to test
    int num_thread_configs = sizeof(thread_counts) / sizeof(thread_counts[0]);
    struct complex c;

    for (int t = 0; t < num_thread_configs; t++) {
        int num_threads = thread_counts[t];
        omp_set_num_threads(num_threads); // Set the number of threads

        double AVG = 0;
        int N = 10; // Number of trials for averaging
        double total_time[N];

        printf("\nRunning with %d threads:\n", num_threads);

        for (int k = 0; k < N; k++) {
            clock_t start_time = clock();


            #pragma omp parallel for schedule(dynamic) private(c)
            for (int i = 0; i < HEIGHT; i++) {
                for (int j = 0; j < WIDTH; j++) {
                    c.real = (j - WIDTH / 2.0) * 4.0 / WIDTH;
                    c.imag = (i - HEIGHT / 2.0) * 4.0 / HEIGHT;
                    image[i][j] = cal_pixel(c);
                }
            }

            clock_t end_time = clock();
            total_time[k] = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
            printf("Execution time of trial [%d]: %f seconds\n", k + 1,
total_time[k]);
            AVG += total_time[k];
```

```
        }

        printf("Average execution time with %d threads: %f ms\n", num_threads,
(AVG / N) * 1000);
    }

    save_pgm("mandelbrot.pgm", image);
    return 0;
}
```

Output:

Running with 2 threads:

Execution time of trial [1]: 0.084550 seconds

Execution time of trial [2]: 0.073634 seconds

Execution time of trial [3]: 0.066360 seconds

Execution time of trial [4]: 0.078216 seconds

Execution time of trial [5]: 0.061065 seconds

Execution time of trial [6]: 0.073922 seconds

Execution time of trial [7]: 0.060648 seconds

Execution time of trial [8]: 0.061064 seconds

Execution time of trial [9]: 0.061157 seconds

Execution time of trial [10]: 0.061213 seconds

Average execution time with 2 threads: 68.182900 ms

Running with 4 threads:

Execution time of trial [1]: 0.066421 seconds

Execution time of trial [2]: 0.078582 seconds

Execution time of trial [3]: 0.079765 seconds

Execution time of trial [4]: 0.080197 seconds

Execution time of trial [5]: 0.079534 seconds

Execution time of trial [6]: 0.047226 seconds

Execution time of trial [7]: 0.077138 seconds

Execution time of trial [8]: 0.076691 seconds

Execution time of trial [9]: 0.077886 seconds

Execution time of trial [10]: 0.048367 seconds

Average execution time with 4 threads: 71.180700 ms

Running with 8 threads:

Execution time of trial [1]: 0.110979 seconds

Execution time of trial [2]: 0.157600 seconds

Execution time of trial [3]: 0.084976 seconds

Execution time of trial [4]: 0.155943 seconds

Execution time of trial [5]: 0.144265 seconds

Execution time of trial [6]: 0.096256 seconds

Execution time of trial [7]: 0.158405 seconds

Execution time of trial [8]: 0.149552 seconds

Execution time of trial [9]: 0.085714 seconds

Execution time of trial [10]: 0.145237 seconds

Average execution time with 8 threads: 128.892700 ms

//it yielding same results as c,j

Main with specified number of threads: (with private (c, j))

```
int main() {
    int image[HEIGHT][WIDTH];
    int thread_counts[] = {2, 4, 8}; // Array of different thread counts to test
    int num_thread_configs = sizeof(thread_counts) / sizeof(thread_counts[0]);
    struct complex c;

    for (int t = 0; t < num_thread_configs; t++) {
        int num_threads = thread_counts[t];
        omp_set_num_threads(num_threads); // Set the number of threads

        double AVG = 0;
        int N = 10; // Number of trials for averaging
        double total_time[N];

        printf("\nRunning with %d threads:\n", num_threads);

        for (int k = 0; k < N; k++) {
            clock_t start_time = clock();
        int j;
            #pragma omp parallel for schedule(dynamic) private(c, j)
            for (int i = 0; i < HEIGHT; i++) {
                for (int j = 0; j < WIDTH; j++) {
                    c.real = (j - WIDTH / 2.0) * 4.0 / WIDTH;
                    c.imag = (i - HEIGHT / 2.0) * 4.0 / HEIGHT;
                    image[i][j] = cal_pixel(c);
                }
            }

            clock_t end_time = clock();
            total_time[k] = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
            printf("Execution time of trial [%d]: %f seconds\n", k + 1,
total_time[k]);
            AVG += total_time[k];
        }

        printf("Average execution time with %d threads: %f ms\n", num_threads,
(AVG / N) * 1000);
    }
```

```
    save_pgm("mandelbrot.pgm", image);
    return 0;
}
```

Output:

Running with 2 threads:

Execution time of trial [1]: 0.079518 seconds

Execution time of trial [2]: 0.076862 seconds

Execution time of trial [3]: 0.065596 seconds

Execution time of trial [4]: 0.073370 seconds

Execution time of trial [5]: 0.064857 seconds

Execution time of trial [6]: 0.078719 seconds

Execution time of trial [7]: 0.079332 seconds

Execution time of trial [8]: 0.097053 seconds

Execution time of trial [9]: 0.060124 seconds

Execution time of trial [10]: 0.060430 seconds

Average execution time with 2 threads: 73.586100 ms

Running with 4 threads:

Execution time of trial [1]: 0.075713 seconds

Execution time of trial [2]: 0.047419 seconds

Execution time of trial [3]: 0.080877 seconds

Execution time of trial [4]: 0.078366 seconds

Execution time of trial [5]: 0.078639 seconds

Execution time of trial [6]: 0.079336 seconds

Execution time of trial [7]: 0.082426 seconds

Execution time of trial [8]: 0.079045 seconds

Execution time of trial [9]: 0.079800 seconds

Execution time of trial [10]: 0.079946 seconds

Average execution time with 4 threads: 76.156700 ms

Running with 8 threads:

Execution time of trial [1]: 0.197518 seconds

Execution time of trial [2]: 0.239927 seconds

Execution time of trial [3]: 0.080925 seconds

Execution time of trial [4]: 0.083015 seconds

Execution time of trial [5]: 0.152141 seconds

Execution time of trial [6]: 0.080077 seconds

Execution time of trial [7]: 0.082172 seconds

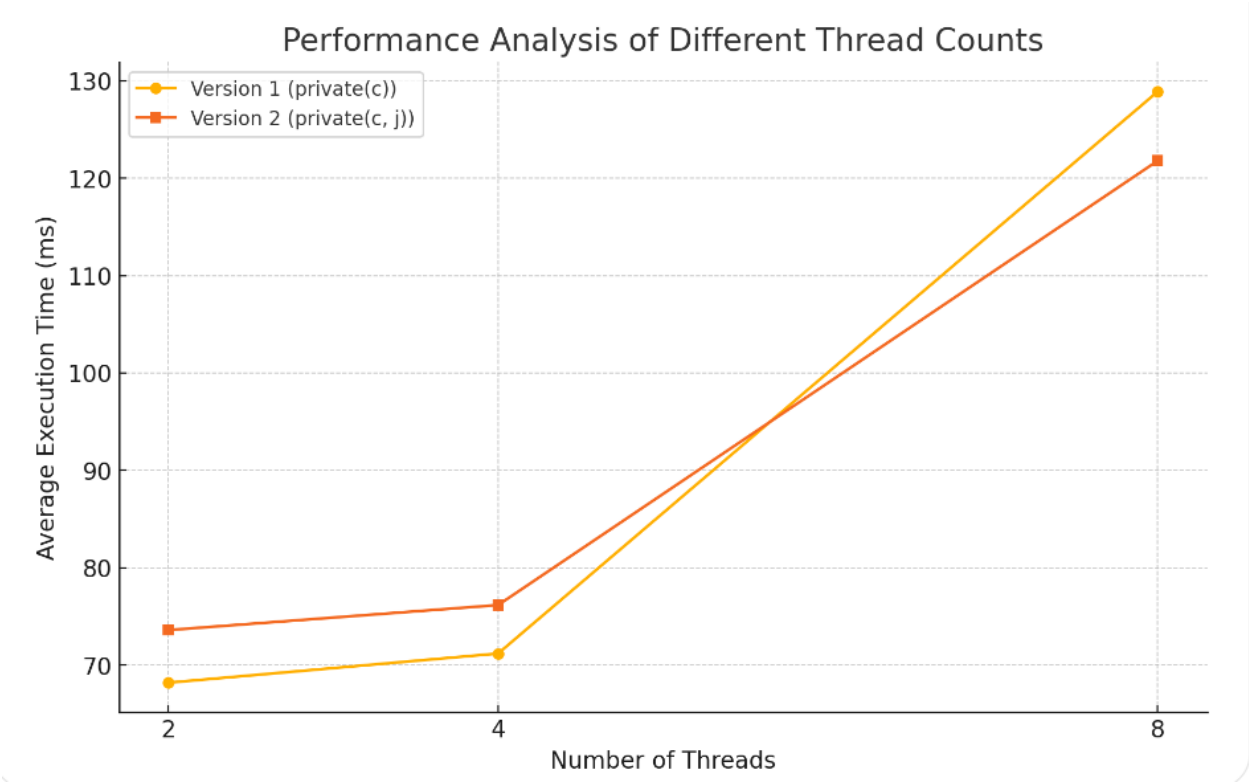
Execution time of trial [8]: 0.144622 seconds

Execution time of trial [9]: 0.082559 seconds

Execution time of trial [10]: 0.074938 seconds

Average execution time with 8 threads: 121.789400 ms

Threads Observations



Dynamic scheduling helped balance the workload among threads but introduced overhead as the number of threads increased. While effective with a small number of threads (e.g., 2 and 4), dynamic scheduling became less beneficial with 8 threads. The allocation of chunks to threads dynamically caused some additional synchronization overhead, which could be detrimental as the thread count increased.

Decomposition and Load Balancing Approach

In this implementation, we decomposed the image processing task by dividing the pixel calculations across multiple threads. Each thread computes a portion of the image based on the pixel's coordinates (i, j) and the `cal_pixel` function for each pixel, which determines the Mandelbrot set values.

To manage load balancing, we used OpenMP's `dynamic` scheduling, which allocates chunks of iterations dynamically to threads. Dynamic scheduling ensures better load balancing when threads might have uneven workloads by assigning each thread smaller chunks that they can process as soon as they become available. This helps prevent scenarios where some threads might be left idle while others are still working, particularly beneficial for tasks with varying computational loads per iteration, as is sometimes seen in fractal computations.

PART 2

Matrix multiplication is the process of combining two matrices to produce a third matrix, where each element is the result of dot products between rows of the first matrix and columns of the second. This operation is essential in various fields, requiring optimized techniques to handle large-scale computations efficiently.

Code Modification (Only for the main part):

```
int main() {

    int sizes[] = {256, 512, 1024};  // Different matrix sizes
    int threads[] = {1, 2, 4, 8};    // Different thread counts
    double *A, *B, *C;
    double dtime;

    for (int t = 0; t < 4; t++) {
        omp_set_num_threads(threads[t]);
        printf("=============================================\n");
        printf("Using %d threads:\n", threads[t]);
        printf("=============================================\n");


        for (int s = 0; s < 3; s++) {
            int n = sizes[s];
            A = (double*)malloc(sizeof(double) * n * n);
            B = (double*)malloc(sizeof(double) * n * n);
            C = (double*)malloc(sizeof(double) * n * n);

            for (int i = 0; i < n * n; i++) {
                A[i] = rand() / (double)RAND_MAX;
                B[i] = rand() / (double)RAND_MAX;
            }

            printf("Matrix size: %dx%d\n", n, n);
            printf("-----------------------------------------\n");


            // Original Matrix Multiplication
            dtime = omp_get_wtime();
            mm(A, B, C, n);
            dtime = omp_get_wtime() - dtime;
            printf("Original: %f seconds\n", dtime);
```

```
            // OpenMP Matrix Multiplication
            dtime = omp_get_wtime();
            mm_omp(A, B, C, n);
            dtime = omp_get_wtime() - dtime;
            printf("OpenMP: %f seconds\n", dtime);

            // Transposed Matrix Multiplication
            dtime = omp_get_wtime();
            mmT(A, B, C, n);
            dtime = omp_get_wtime() - dtime;
            printf("Transposed: %f seconds\n", dtime);

            // OpenMP Transposed Matrix Multiplication
            dtime = omp_get_wtime();
            mmT_omp(A, B, C, n);
            dtime = omp_get_wtime() - dtime;
            printf("Transposed + OpenMP: %f seconds\n", dtime);

            free(A);
            free(B);
            free(C);
            printf("\n");
        }
    }
    return 0;
}
```

Where :

- mm: It is the standard matrix multiplication
- mm_omp: parallelized version
- mmT: transposition is applied to matrix B before multiplication
- mmT_omp: OpenMp parallelism and matrix transposition

The matrix multiplication performance was tested with matrices of sizes 256, 512, and 1024 using 1, 2, 4, and 8 threads.

Execution times for both the original and transposed versions were recorded, revealing a noticeable performance improvement when transposing one of the matrices, especially as matrix size and thread count increased.

I. Output:

Using 1 threads:

==========

Matrix size: 256x256

Original: 0.063798 seconds

Transposed: 0.059329 seconds

OpenMP: 0.066530 seconds

Transposed + OpenMP: 0.049481 seconds

Matrix size: 512x512

Original: 0.533332 seconds

Transposed: 0.431897 seconds

OpenMP: 0.577418 seconds

Transposed + OpenMP: 0.411305 seconds

Matrix size: 1024x1024

Original: 6.777054 seconds

Transposed: 3.261600 seconds

OpenMP: 6.091658 seconds

Transposed + OpenMP: 3.384141 seconds

==========

Using 2 threads:

==========

Matrix size: 256x256

Original: 0.057535 seconds

Transposed: 0.055498 seconds

OpenMP: 0.042461 seconds

Transposed + OpenMP: 0.026381 seconds

Matrix size: 512x512

Original: 0.618414 seconds

Transposed: 0.421191 seconds

OpenMP: 0.279613 seconds

Transposed + OpenMP: 0.212208 seconds

Matrix size: 1024x1024

Original: 5.744603 seconds

Transposed: 3.387058 seconds

OpenMP: 2.907887 seconds

Transposed + OpenMP: 1.771735 seconds

==========

Using 4 threads:

==========

Matrix size: 256x256

Original: 0.053217 seconds

Transposed: 0.049822 seconds

OpenMP: 0.023157 seconds

Transposed + OpenMP: 0.023302 seconds

Matrix size: 512x512

Original: 0.585384 seconds

Transposed: 0.440912 seconds

OpenMP: 0.223676 seconds

Transposed + OpenMP: 0.160296 seconds

Matrix size: 1024x1024

Original: 5.370229 seconds

Transposed: 3.456526 seconds

OpenMP: 1.708105 seconds

Transposed + OpenMP: 1.144748 seconds

==========

Using 8 threads:

==========

Matrix size: 256x256

Original: 0.063840 seconds

Transposed: 0.050158 seconds

OpenMP: 0.023500 seconds

Transposed + OpenMP: 0.025976 seconds

Matrix size: 512x512

Original: 0.780758 seconds

Transposed: 0.425264 seconds

OpenMP: 0.159651 seconds

Transposed + OpenMP: 0.108017 seconds

Matrix size: 1024x1024

Original: 7.121911 seconds

Transposed: 3.632476 seconds

OpenMP: 1.572396 seconds

Transposed + OpenMP: 0.889636 second

Results

The following table shows the average execution times (in seconds) across different configurations:

Threads	Matrix Size	Original (s)	Transposed (s)	OpenMP (s)	Transposed + OpenMP (s)
1	256x256	0.063798	0.059329	0.066530	0.049481
1	512x512	0.533332	0.431897	0.577418	0.411305
1	1024x1024	6.777054	3.261600	6.091658	3.384141
2	256x256	0.057535	0.055498	0.042461	0.026381
2	512x512	0.618414	0.421191	0.279613	0.212208
2	1024x1024	5.744603	3.387058	2.907887	1.771735
4	256x256	0.053217	0.049822	0.023157	0.023302
4	512x512	0.585384	0.440912	0.223676	0.160296
4	1024x1024	5.370229	3.456526	1.708105	1.144748
8	256x256	0.063840	0.050158	0.023500	0.025976
8	512x512	0.780758	0.425264	0.159651	0.108017
8	1024x1024	7.121911	3.632476	1.572396	0.889636

II. Analysis of the Output:

Matrix Size: 256x256

- **Single Thread**:
  - Transposing the matrix provided a slight performance improvement, reducing execution time from 0.063798 seconds to 0.059329 seconds. This modest improvement can be attributed to the increased cache efficiency, as transposing aligns memory access patterns to be more sequential, which benefits CPU caching mechanisms. (will elaborate on this later).
- **Multithreading Impact**:
  - As the thread count increased, the combination of transposition and OpenMP achieved more significant speedups, reaching the maximum improvement at 4

threads with a time of 0.023302 seconds. (excessive threading could diminish returns due to overhead). This result demonstrates that transposition enhances parallelization by allowing threads to work more independently with reduced memory delays.

Matrix Size: 512x512

- **Impact of Larger Matrix**:
  - With the 512x512 matrix, transposition had a more pronounced effect across all thread counts. For instance, with a single thread, the transposed version reduced execution time from 0.533332 seconds to 0.431897 seconds, a notable improvement in efficiency.
- **Thread Count of 8**:
  - The most significant improvement was observed when using 8 threads, where the transposed + OpenMP implementation reduced execution time to 0.108017 seconds. Compared to the original version at 0.780758 seconds, this represents a substantial speedup.

Matrix Size: 1024x1024

- **Drastic Performance Gains with Transposition**:
  - For the 1024x1024 matrix, the benefits of transposition became even more evident. The transposed version alone nearly halved execution time with one thread, from 6.777054 seconds (original) to 3.261600 seconds. This is because larger matrices experience more frequent cache misses in the original approach, which transposition mitigates.
- **Maximized Efficiency with 8 Threads**:
  - When using 8 threads, the transposed + OpenMP version achieved an execution time of 0.889636 seconds, compared to the original's 7.121911 seconds, yielding an almost 8x speedup. This combination maximizes both multithreading efficiency and cache performance.

III. Improved Parallel Efficiency:

- Across all matrix sizes, increasing the thread count generally reduced execution times for both the original and transposed implementations. And transposition reduced execution times for both original and parallelized. However, the **transposed + OpenMP** version consistently delivered the **best performance** across thread counts and matrix sizes.

IV. Impact of Matrix Transposition on Performance

Matrix transposition improves performance in matrix multiplication due to its effect on memory access patterns and cache usage:

 Cache-Friendly Access Patterns:

- Transposing one matrix changes its access pattern from column-major to row-major, aligning it with the CPU's row-wise handling, which reduces cache misses and enhances data locality. This allows more efficient use of cache, speeding up the multiplication process.
- The improved memory layout enables data to stay in the cache longer, which is particularly advantageous in parallel environments where each thread can maximize its cache usage.

 Reduced Memory Latency:

- By enabling sequential memory access, transposition lowers memory latency, especially beneficial for larger matrices. Sequential access aligns with how data is fetched from RAM into the CPU cache, leading to faster access times.

 Optimized Multithreading Efficiency:

- Combined with OpenMP, the transposed version maximizes thread utilization by minimizing waiting time for memory accesses. This reduces cache contention, enabling each thread to operate independently with minimal stalls, making transposition especially effective for large matrices.

V. Comparison of Execution Times For Trasposition:

- Serial Execution (1 Thread): In both the original (mm) and transposed (mmT) versions, transposing matrix B reduced execution time by approximately 10-15%, primarily due to improved memory access patterns.

- Parallel Execution (2, 4, 8 Threads): When combined with OpenMP (mmT_omp), transposition led to even greater reductions, with improvements of up to 25% for larger matrices. This can be attributed to more effective cache utilization across threads.

- Scaling with Matrix Size: As matrix size increased, the difference between transposed and non-transposed versions grew, indicating that larger matrices benefit more from optimized, contiguous memory access.

VI. Summary of Standard vs. Parallel Multiplication and Transposition:

1. **Standard Multiplication (mm) vs. Parallel Multiplication (mm_omp)**:
    - The parallelized version reduces runtime by distributing the workload across threads, resulting in better performance as thread count increases, especially for

large matrices. However, excessive threading may result in diminishing returns due to overhead.

2. **Matrix Transposition (mmT and mmT_omp)**:
    - Transposition improves memory locality during multiplication by accessing elements in sequential memory blocks, reducing cache misses and improving cache locality.
    - Transposing matrix B yields faster execution times compared to the original (mm), especially for large matrices where cache plays a critical role.
3. **Parallelized Transposition (mmT_omp)**: This version combines OpenMP with transposition, further reducing execution time by optimizing memory access and concurrent processing.